

Tree-based method: Bagging and boosting

Advantages and disadvantages with tree-based methods

- ▶ (+) Easy to understand and interpret
- ▶ (+) The result can be interpreted as a graph
- ▶ (+) Can handle categorical variables without a need to create dummy variables
- ▶ (-) Predictions are often not very good
- ▶ (-) The estimated model can change a lot by small changes in the dataset

Bagging (Bootstrap aggregation)

- ▶ Variance of averages: $X_1, X_2, \dots, X_n$ iid with $\text{Var}(X) = \sigma^2$

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \underbrace{\sum_{i=1}^n \text{Var}(X_i)}_{n\sigma^2} = \frac{\sigma^2}{n}$$



$$\therefore \text{Var}(\bar{X}) < \text{Var}(X) \text{ if } n > 1$$

- ▶ Averaging reduces variance

Bagging

Example: We shall predict Y with KNN. To exemplify we generate data from the following data generating process

$$Y = 10 + 0.1X + (X - 0.2)^2 + (X - 0.4)^3 + \varepsilon,$$

where $\varepsilon \sim N(0, 0.3^2)$

Can the KNN predictions be improved?

Idea: Reduce prediction error variance by predicting from many training data sets and average the predictions.

But we have only one training data set?

Bagging for KNN

- ▶ Training data: $(x_1, y_1), \dots, (x_n, y_n)$
- ▶ Predict y_0 based on x_0
- ▶ Resample training data with replacement

1. Resample from the training data $\implies$

$$(x_1^{(1)}, y_1^{(1)}), \dots, (x_n^{(1)}, y_n^{(1)})$$

2. Predict y_0 with the resampled data by KNN $\implies \hat{f}^{*1}(x_0)$
3. Repeat 2, B times to obtain $\hat{f}^{*1}(x_0), \hat{f}^{*2}(x_0), \dots, \hat{f}^{*B}(x_0)$
4. Predict y_0 by

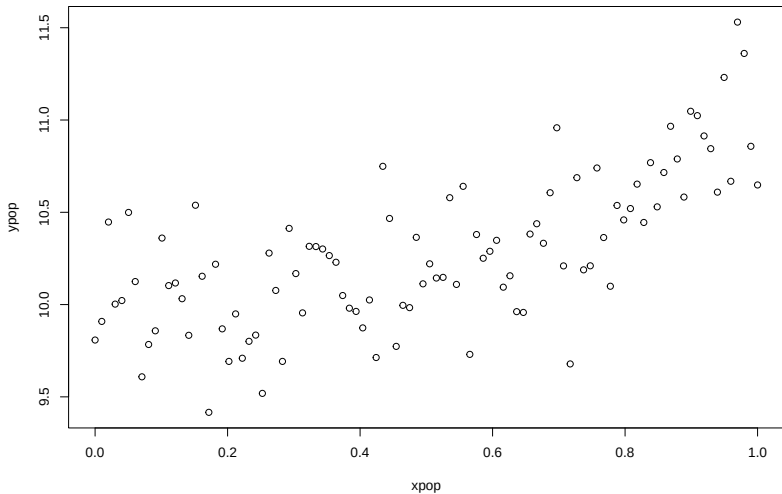
$$\hat{f}_{avg}(x_0) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x_0)$$

An example

This is how the data is generated (of course unknown in a practical application)

```
n=100
set.seed(123)
xpop=seq(0,1,len=n)
ypop=10+0.1*xpop+(xpop-0.2)^2+(xpop-0.4)^3+0.3*rnorm(n)
itrain=sample(1:n,size=floor(n/2))
xtrain=xpop[itrain]
ytrain=ypop[itrain]
xtest=xpop[-itrain]
ytest=ypop[-itrain]
```

An example



An example

To exemplify, we use KNN with $K = 3$

```
knn=function(x0,x,y,K=sqrt(length(x)))  
{  
  dist=abs(x-x0)  
  odist=order(dist)  
  y0=mean(y[odist[1:K]])  
  return(y0)  
}
```


An example

In this example, we use the validation set approach to estimate the testMSE

```
pred1=apply(as.matrix(xtest),1,knn,x=xtrain,y=ytrain,K=3)
mse1=mean((ytest-pred1)^2)
mse1
```

```
## [1] 0.06069237
```

An example

```
ntrain=floor(n/2)
B=100
pred=matrix(0,B,n)
for(b in 1:B)
{
  ind=sample(1:ntrain,size=ntrain,replace = TRUE)
  xb=xtrain[ind]
  yb=ytrain[ind]
  pred[b,]=apply(as.matrix(xtest),1,knn,x=xb,y=yb)
}
predbag=apply(pred,2,mean)
mse1
```

```
## [1] 0.06069237
```

```
msebag=mean((ytest-predbag)^2)
msebag
```

```
## [1] 0.05637494
```

KNN and bagging

- ▶ KNN is a good example to illustrate bagging since it is a simple method and demands little R-code to run
- ▶ However, it is not the method where the benefit is greatest
- ▶ The reason for this is that KNN is a relatively stable method, i.e., a small change in the dataset will likely not change the prediction much.
- ▶ Regression and classification trees, on the other hand, is unstable and benefit more from bagging

Bagging for tree-based methods

- ▶ Training data: $(x_1, y_1), \dots, (x_n, y_n)$

Predict y_0 based on x_0

Resample training data with replacement

1. Resample from the training data $\implies$

$$(x_1^{(1)}, y_1^{(1)}), \dots, (x_n^{(1)}, y_n^{(1)})$$

2. Fit the tree to the resampled data

3. Predict y_0 with the tree $\implies \hat{f}^{*1}(x_0)$

Bagging for tree-based methods

4. Repeat 1-3, B times to obtain $\hat{f}^{*1}(x_0), \hat{f}^{*2}(x_0), \dots, \hat{f}^{*B}(x_0)$
5. Predict y_0 by

$$\hat{f}_{avg}(x_0) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x_0)$$

- If classification problems: The category with the plurality of the B predictions.

The Carseat data

First a simple regression tree to predict Sales. Evaluated by the validation set approach

```
library(ISLR)
library(tree)
n=nrow(Carseats)
ind=sample(1:n,size=floor(n/2))
train=Carseats[ind,]
test=Carseats[-ind,]
tree1=tree(Sales~.,data=train)
pred1=predict(tree1,newdata=test)
mse1=mean((test$Sales-pred1)^2)
mse1
```

```
## [1] 4.881557
```

The Carseat data - bagged regression tree

```
library(randomForest)
bagging=randomForest(Sales~.,data=train,mtry=10)
predbag=predict(bagging,newdata=test)
msebag=mean((test$Sales-predbag)^2)
msebag
```

```
## [1] 3.113909
```

Random forests

- ▶ Random forests is an extension of bagging trees in that we, for each bootstrap sample, we choose a subset of the predictors. That is, we do not allow for all predictors to be included in the trees.
- ▶ A consequence of this is that the predictions from the different trees will be less correlated compared to the case in bagging.
- ▶ This in turn will reduce the variance of the average prediction from the trees.

Random forests

Training data: $(x_1, y_1), \dots, (x_n, y_n)$

Predict y_0 based on x_0

Resample training data with replacement

1. Resample from the training data $\implies$

$$(x_1^{(1)}, y_1^{(1)}), \dots, (x_n^{(1)}, y_n^{(1)})$$

2. **For each resample, out of the p predictors, choose $m < p$ randomly**
3. Fit the tree to the resampled data using the m predictors
4. Predict y_0 with the tree $\implies \hat{f}^{*1}(x_0)$
5. Repeat 1-4, B times to obtain $\hat{f}^{*1}(x_0), \hat{f}^{*2}(x_0), \dots, \hat{f}^{*B}(x_0)$
6. Predict y_0 by

$$\hat{f}_{avg}(x_0) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x_0)$$

The Carseat data - Random Forest

```
library(randomForest)
rforest=randomForest(Sales~.,data=train,mtry=3)
predrf=predict(rforest,newdata=test)
mse1=mean((test$Sales-predrf)^2)
mse1
```

```
## [1] 4.881557
```

```
msebag
```

```
## [1] 3.113909
```

```
mserf
```

```
## [1] 3.568976
```

A few more words about bagged trees

- ▶ Bagging (bootstrap aggregation) is the idea of resampling the training data, re-predict on the test data and finally average, or, for classification, compute majority vote, of the predictions.
- ▶ The effect of averaging is to reduce the variance of the predictions.
- ▶ Since averaging uncorrelated variables reduces the variance more than averaging correlated variables, a trick to decorrelate the predictions can be used.
- ▶ This method is called Random Forests and is the same as bagged trees but with the addition that a randomly selected subset of the predictors (X-variables) is used for each bootstrap sample.
- ▶ The effect of this is a trade-off between sometimes obtaining regression- or classification trees which exclude potentially good predictors but, on the other hand, get less correlated predictions.

Out-of-bag error estimation

- ▶ A very useful side effect with bagging is that we get a computationally efficient way of doing cross-validation.
- ▶ This is called out-of-bag (OUB) error estimation.
- ▶ We have n observations and would like to predict Y using bagged regression trees and the predictors $X_1, \dots, X_p$.
- ▶ In bootstrap sample number b have not sampled observation number m . This means that we can see Y_m as test data for this regression tree.
- ▶ As it turns out, in around one third, $B/3$ of the bootstrap samples, this observation will not be present. For each of these, we can average the predictions of Y_m .
- ▶ Such test predictions can be done for each of the n observations and consequently, e.g., a testMSE, can be computed.

Variable importance

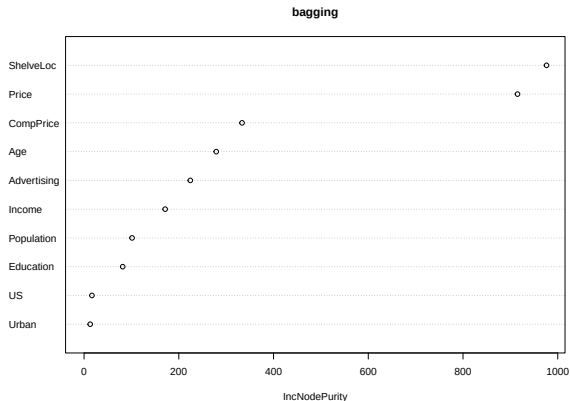
- ▶ One argument for using regression- and classification trees is that we are able to visualize which variables that help us in predicting the dependent variable.
- ▶ The argument for using bagging and Random Forests is that we improve the predictions, which is often not very impressive for regression- and classification trees. However, the predictions are computed by many regression trees and it is therefore not immediately clear which variables are contributing to predictive accuracy.
- ▶ We would like to measure *variable importance* for the predictors.

Variable importance

- ▶ In a regression tree, a measure of how important a predictor variable is, is how much it reduces the RSS over all splits (a variable can contribute to splits in several branches of the tree).
- ▶ In bagging and Random Forests, the reduced RSS is summed over all splits and all trees for each variable.
- ▶ For classification trees reduction of the Gini index is considered rather than RSS.

Example with the Carseat data

```
library(ISLR)
library(randomForest)
bagging=randomForest(Sales~.,data=Carseats,mtry=10)
varImpPlot(bagging)
```



Boosted trees

- ▶ Boosting is similar to bagging in the sense that it is averaging predictions over many trees.
- ▶ It is different in the sense that it fit the trees sequentially; in bagging each tree is fitted to a bootstrapped sample, independently of the other trees.
- ▶ In boosting, the trees are sequentially fitted to a modified version of the dataset which is updated based on the previous trees ability to predict the different observations.

Boosted trees

The algorithm in ISLR, Ch 8:

1. Initialize the prediction by $\hat{f}(x) = 0$. Also, initialize $r_i = y_i$ for all training observations $i = 1, \dots, n$.
2. For $b = 1, \dots, B$:
 - a) Fit a tree, $\hat{f}_b$ with d splits to the training data (X, r) .
 - b) Update the prediction by

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}_b(x)$$

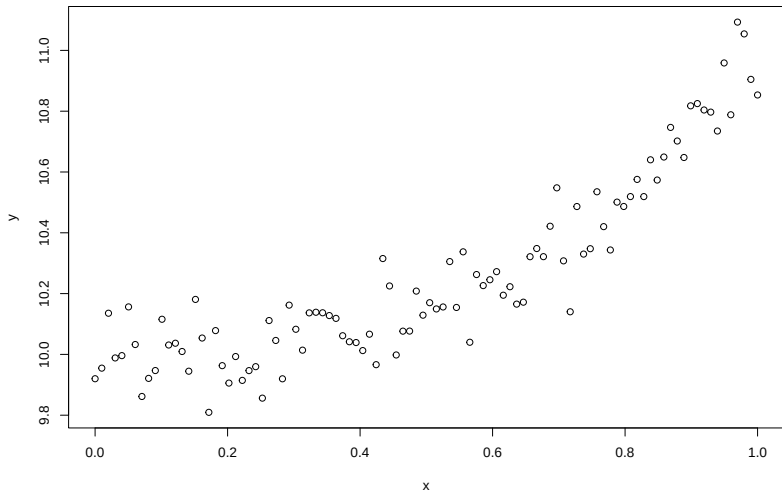
- c) Update the residuals by

$$r_i \leftarrow r_i - \lambda \hat{f}_b(x_i)$$

3. The prediction is

$$\hat{f}(x) = \lambda \sum_{b=1}^B \hat{f}_b(x)$$

Example one



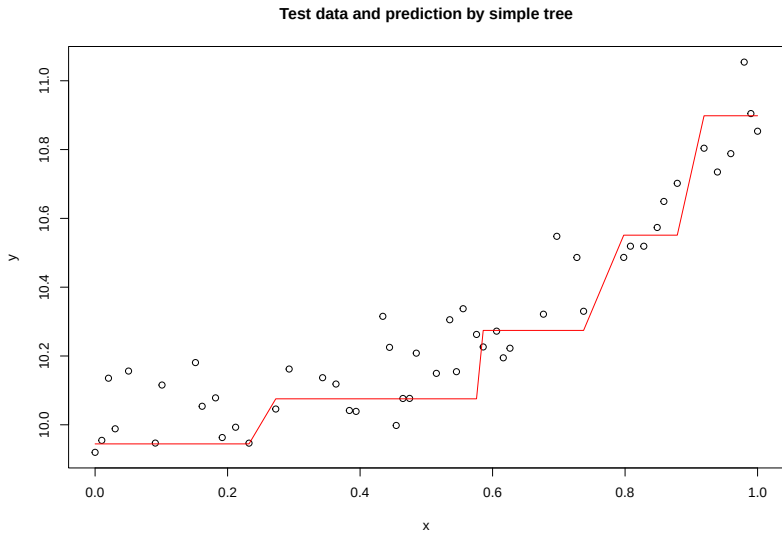
Example one

```
set.seed(123)
# TRAINING AND TEST DATA
itrain=sample(1:n,size=floor(n/2))
xtrain=xpop[itrain]
ytrain=ypop[itrain]
xtest=ypop[-itrain]
ytest=ypop[-itrain]
traindata=data.frame(x=xtrain,y=ytrain)
testdata=data.frame(x=xtest,y=ytest)
```

First, a standard regression tree

```
# SIMPLE TREE  
library(tree)  
treesimple=tree(y~x,data=traindata)  
pred1=predict(treesimple,newdata=testdata)  
mse1=mean((ytest-pred1)^2)  
mse1  
  
## [1] 0.01478809
```

First, a standard regression tree



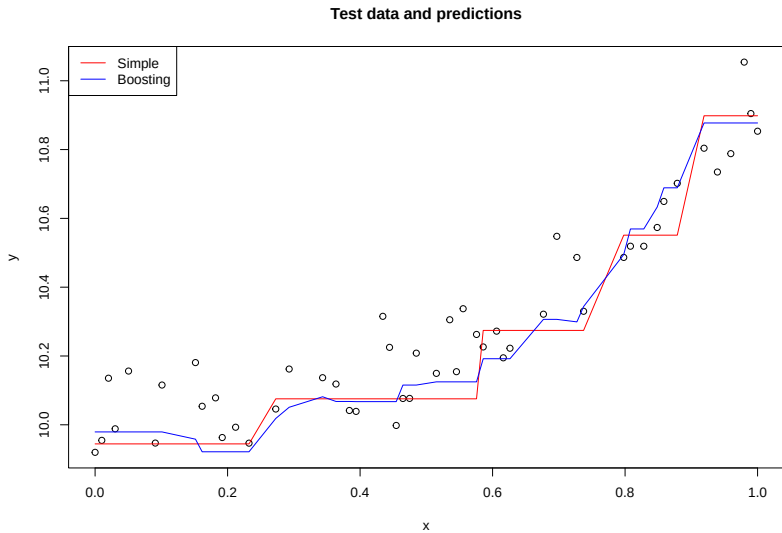
Boosted trees

```
B=1000
lambda=0.01
pred=matrix(0,B,floor(n/2))
rtrain=ytrain
for(b in 2:B)
{
  traindata=data.frame(x=xtrain,y=rtrain)
  tree1=tree(y~x,data=traindata)
  fb=predict(tree1,newdata=testdata)
  pred[b,]=pred[b-1,]+lambda*fb
  fbtrain=predict(tree1)
  rtrain=rtrain-lambda*fbtrain
}
predboost=pred[B,]
mse1
```

```
## [1] 0.01478809
mseboost=mean((ytest-predboost)^2)
mseboost
```

```
## [1] 0.01157523
```

Boosted trees



Boosting

$$\text{MSE}_{\text{simple}} = 0.0147881$$

$$\text{MSE}_{\text{boost}} = 0.0115752$$

An example with one single variable. Let us investigate how the method works on the Hitters data.

The Hitters data

```
set.seed(1234)
library(ISLR)
# REMOVE OBSERVATIONS WITH MISSING SALARY VALUES
head(Hitters$Salary)
```

```
## [1]      NA 475.0 480.0 500.0  91.5 750.0
```

```
head(is.na(Hitters$Salary))
```

```
## [1]  TRUE FALSE FALSE FALSE FALSE FALSE
```

```
Hitters=Hitters[!is.na(Hitters$Salary),]
Hitters$logSalary=log(Hitters$Salary)
n=nrow(Hitters)
```

The Hitters data

```
# TRAINING AND TEST DATA  
itrain=sample(1:n,size=floor(n/2))  
traindata=Hitters[itrain,]  
testdata=Hitters[-itrain,]  
ytest=testdata$logSalary
```

A simple tree

```
formula1=logSalary~CRuns+CHits+HmRun+CWalks+CRBI+Hits+RBI  
treesimple=tree(formula1,data=traindata)  
pred1=predict(treesimple,newdata=testdata)  
mse1=mean((ytest-pred1)^2)  
mse1
```

```
## [1] 0.3851602
```

Boosting

```
## USING gbm
library(gbm)

## Loaded gbm 2.2.2

## This version of gbm is no longer under development. Consider

boost=gbm(formula1,data=Hitters[itrain,],distribution
           ="gaussian",n.trees=1000,interaction.depth=4,
           shrinkage=0.01)
predgbm=predict(boost,newdata=Hitters[-itrain,],n.trees=1000)
msegbm=mean((ytest-predgbm)^2)
msegbm

## [1] 0.3591336
```

Boosting

$$\text{MSE}_{\text{simple}} = 0.3851602$$

$$\text{MSE}_{\text{boost}} = 0.3591336$$

Works here too